Member 2: Fraud and Diagnosis Intelligence

Work assignment for the second student in the 3-member team

UPI Payment Failure Diagnosis, Reconciliation and Fraud Risk Monitoring System

Your job in one line: build the two smart parts of the system. Six concurrent fraud checks plus a trained model decide how risky a payment is. A rule table decides why a stuck payment is stuck and what to do. They are different kinds of problems, so you build them with different methods and keep them apart.

1. Your Role and How It Fits

Member	Track	Builds
1	backend/	Database, payment APIs, state machine, idempotency, bank simulators
2 (you)	fraud-intel/	Six threaded fraud checks, fraud ML model, diagnosis decision table, recovery-time estimate
3	frontend/	UPI app screens, operations dashboard, alerts, end-to-end testing

Member 1's payment flow calls your code at two points: before the debit (fraud score) and whenever a payment is stuck (diagnosis and recovery estimate). Member 3 shows your results on screen. You do **not** change payment states or write UI code.

> **Keep these two apart. The six threads and the ML model answer: is this payment suspicious? The rule table answers: why is this payment stuck? Never say the six threads find the fault, because the rules do.**

2. What You Deliver

- Six independent fraud checks, each returning a score from 0 to 100 and a reason.
- A shared thread pool that runs the six checks at the same time, with thread name and start/end time logged.
- A benchmark comparing sequential and concurrent runs on the same batch (your concurrency proof for the viva).
- A decision table that turns real-time signals into a reason, next state, action, user message and severity.
- A recovery-time estimator for stuck payments, always shown as an estimate.
- A synthetic fraud dataset, a baseline model (Logistic Regression) and a final model (CatBoost), evaluated properly.
- One function, **score_transaction(txn)**, that Member 1 calls and that returns a decision and score.
- Tests for every case in section 8.

3. Setup

```
pip install scikit-learn catboost pandas numpy pytest
```

Suggested layout inside fraud-intel/:

```
fraud-intel/
  fraud_intel/
    checks.py        the six checks
    scoring.py       thread pool, aggregation, score_transaction()
    benchmark.py     sequential vs concurrent timing
    diagnosis.py     rule table and diagnose()
    recovery.py      recovery-time estimator
  ml/
    make_data.py     synthetic dataset generator
    train.py         baseline and CatBoost training
    model.py         load model, build features, predict
  tests/             pytest files
```

4. Suggested Timeline

When	Phase	Your tasks	Done when
Days 1-2	0 Design lock	Agree function signatures, context fields and signal names with Member 1	Both sides wrote them down
Week 1	1 Skeleton	Steps 1-2: six checks run one after another, diagnosis table	Checks return scores; diagnose() passes its tests
Week 2	2 Happy path	Hook score_transaction into Member 1's flow	A real payment gets a real decision
Week 3	3 Core	Steps 2-3 and 5-6: thread pool and benchmark, recovery estimate, dataset, models	Benchmark numbers recorded; models trained and evaluated
Week 4	4 Hardening	Step 7-8: blend model into scoring, tests, final numbers	All cases in section 8 pass
Week 5	5 Polish	Report results section, explain every number in the viva	Guide's dry-run passes

5. Step-by-Step Tasks

Step 0. Design lock with Member 1 (before coding)

Agree these in writing, in docs/architecture.md:

- **Function signature: score_transaction(txn)** returns **{"decision": "APPROVE|REVIEW|BLOCK", "score": number, "checks": [...]}**. Member 1 only needs decision and score; Member 3 shows the checks.
- **Transaction fields** you receive: id, amount, location, hour, merchant id, device id, receiver id.
- **Context** (customer profile) you need: usual amount, home location, known devices, known receivers, recent transaction counts, typical hours, merchant risk. Decide who loads it from the database. Recommended: Member 1 supplies a helper, and you read the result as plain data (see Step 1 for why).
- **Diagnosis signals:** debit (OK, ERROR, TIMEOUT), credit (OK, ERROR, TIMEOUT), status_check (True, False, None), duplicate, receiver_valid, balance_ok, network_ok, bank_up. Member 1 fills these from the simulators.
- **BLOCK / REVIEW handling:** BLOCK stops the payment before debit; REVIEW lets it continue and opens a case.
- **Recovery history:** ask Member 1 to log, for every stuck payment, the reason and how many minutes it took to resolve (the simulator resolves each failure type after a configurable delay). You build the estimate from these logs.

> **Alignment point to raise now: Member 1's first flow marks any failed debit as FAILED. A debit that times out is different, because the money may have left. Rule R05 in your table sends it to TIMEOUT for a status check. Ask Member 1 to follow your table so both sides agree.**

Step 1. The six fraud checks

Each check is a plain function taking the transaction and the customer context, and returning a score, a reason, the thread name and start/end times. They only **read** data, so they need no locks. Read the context from a prepared dictionary rather than sharing one database session between threads, because sessions are not thread-safe.

Check	Looks at	Score rule (starting point)	Weight
Amount	Amount vs usual amount	up to 1.5x: 0; up to 3x: 40; above: 90	0.25
Location	Location vs home	same: 0; different: 70; missing: 20	0.20
Velocity	Transactions in last 10 min	up to 2: 0; up to 5: 50; more: 85	0.20
Behavior	Hour and amount pattern	both odd: 80; one odd: 40; none: 0	0.15
Merchant	Merchant risk lookup	the merchant's risk level; person-to-person: 0	0.10
Device	Known device or not	known: 0; new: 55; missing: 40	0.10

```
import time
import threading

# Simulated latency (seconds) of the lookup each check makes, e.g. a device
# reputation service or a geo lookup. This waiting is what threads overlap.
LATENCY = {"amount": 0.05, "location": 0.15, "velocity": 0.10,
           "behavior": 0.20, "merchant": 0.08, "device": 0.12}


def _lookup(name):
    time.sleep(LATENCY[name])


def _result(name, score, reason, t0):
    return {"check": name, "score": score, "reason": reason,
            "thread": threading.current_thread().name,
            "start": t0, "end": time.time()}


def check_amount(txn, ctx):
    t0 = time.time(); _lookup("amount")
    ratio = txn["amount"] / max(ctx["usual_amount"], 1)
    if ratio <= 1.5:
        return _result("amount", 0, "Amount close to usual", t0)
    if ratio <= 3:
        return _result("amount", 40, f"{ratio:.1f}x the usual amount", t0)
    return _result("amount", 90, f"{ratio:.1f}x the usual amount, highly unusual", t0)

def check_device(txn, ctx):
    t0 = time.time(); _lookup("device")
    if not txn.get("device_id"):
        return _result("device", 40, "No device information", t0)
    if txn["device_id"] in ctx["known_devices"]:
        return _result("device", 0, "Recognised device", t0)
    return _result("device", 55, "New or unrecognised device", t0)


CHECKS = {"amount": check_amount, "location": check_location,
          "velocity": check_velocity, "behavior": check_behavior,
          "merchant": check_merchant, "device": check_device}
```

Be honest about why threads help. Each check makes a lookup (device reputation, geo, merchant list). In this project those lookups are simulated with a short wait, which stands for network time. Threads overlap that waiting. This is I/O-bound work, so threads fit; CPU-heavy work would need processes.

Step 2. Thread pool and scoring

Create one shared pool at startup. For each payment submit the six checks, collect them as they finish, then combine. The weighted sum maps to the decision: 0 to 30 APPROVE, 31 to 70 REVIEW, 71 to 100 BLOCK.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
from fraud_intel.checks import CHECKS
```

```
from ml.model import ml_probability

WEIGHTS = {"amount": 0.25, "location": 0.20, "velocity": 0.20,
           "behavior": 0.15, "merchant": 0.10, "device": 0.10}
APPROVE_MAX, REVIEW_MAX = 30, 70          # 0-30 approve, 31-70 review, 71-100 block

EXECUTOR = ThreadPoolExecutor(max_workers=10, thread_name_prefix="fraud")


def load_context(txn):
    """Phase 1 stub. Later: read the customer profile from the database."""
    return {"usual_amount": 2000, "home_location": "Hubballi",
            "known_devices": {"dev-1"}, "recent_count": 0,
            "typical_hours": (8, 22), "merchant_risk": 10}


def run_sequential(txn, ctx):
    return [fn(txn, ctx) for fn in CHECKS.values()]


def run_concurrent(txn, ctx):
    futures = [EXECUTOR.submit(fn, txn, ctx) for fn in CHECKS.values()]
    return [f.result() for f in as_completed(futures)]


def aggregate(results):
    total = sum(r["score"] * WEIGHTS[r["check"]] for r in results)
    if total <= APPROVE_MAX:
        decision = "APPROVE"
    elif total <= REVIEW_MAX:
        decision = "REVIEW"
    else:
        decision = "BLOCK"
    return round(total, 1), decision


RULE_SHARE = 0.6        # final = 0.6 x rule score + 0.4 x model score (team decision)


def decide(score):
    if score <= APPROVE_MAX:
        return "APPROVE"
    return "REVIEW" if score <= REVIEW_MAX else "BLOCK"


def score_transaction(txn, ctx=None, concurrent=True):
    ctx = ctx or load_context(txn)
    results = run_concurrent(txn, ctx) if concurrent else run_sequential(txn, ctx)
    results.sort(key=lambda r: list(CHECKS).index(r["check"]))
    rule_total, rule_decision = aggregate(results)
    prob = ml_probability(txn, ctx)
    if prob is None:                                # no model file: rules only
        return {"decision": rule_decision, "score": rule_total, "ml_score": None, "checks": results}
    final = round(RULE_SHARE * rule_total + (1 - RULE_SHARE) * prob * 100, 1)
    return {"decision": decide(final), "score": final, "rule_score": rule_total,
            "ml_score": round(prob * 100, 1), "checks": results}
```

This listing is the final version of the file. Build it in stages: first only the checks and aggregate(), then the thread pool, and add the model import and the blend at the end of the file in Step 7. Until a model exists, score_transaction simply returns the rule score.

Check one result by hand before moving on. For a payment of 9500 against a usual 2000, from a new city, at 3 am, on an unknown device, the checks scored amount 90, location 70, velocity 0, behavior 80, merchant 10, device 55. That gives a weighted rule score of 55, which is REVIEW. Each check ran in its own thread (fraud_0 to fraud_5).

Step 3. Benchmark (your concurrency proof)

Run the same batch of payments sequentially and concurrently and record both times. Sequential time is roughly the sum of the six latencies; concurrent time is roughly the slowest single check.

```
import time
from fraud_intel.scoring import score_transaction, load_context

def make_txns(n):
    return [{"id": f"t{i}", "amount": 1000 + i, "location": "Hubballi", "hour": 14,
             "merchant_id": "m1", "device_id": "dev-1"} for i in range(n)]

def timed(concurrent, txns, ctx):
    t0 = time.time()
    for t in txns:
        score_transaction(t, ctx, concurrent=concurrent)
    return time.time() - t0

if __name__ == "__main__":
    txns = make_txns(20); ctx = load_context(None)
    seq = timed(False, txns, ctx)
    con = timed(True, txns, ctx)
    print(f"transactions: {len(txns)}")
    print(f"sequential : {seq:.2f} s  ({seq / len(txns) * 1000:.0f} ms each)")
    print(f"concurrent : {con:.2f} s  ({con / len(txns) * 1000:.0f} ms each)")
    print(f"speed-up   : {seq / con:.1f}x")
```

Run (20 payments)	Total time	Per payment	Expected from latencies
Sequential	about 14 s	about 700 ms	sum = 50+150+100+200+80+120 = 700 ms
Concurrent	about 4 s	about 200 ms	slowest check = 200 ms
Speed-up	about 3.5x		700 / 200 = 3.5

These were measured on the reference code. Your numbers will differ with your latency settings; record your own, re-run the benchmark on the final checks (not the first draft), and put the table in the report. Also keep the thread log (thread name, start, end per check) as supporting evidence.

Step 4. Diagnosis decision table (rules, no machine learning)

This explains why a payment is stuck using only real-time signals, with no training data. Write it as a table so it can appear in the report as a decision table. The first matching row wins.

Rule	Signals	Reason	Next state	Alerts
R01	duplicate = true	Duplicate request	UNCHANGED	none
R02	receiver invalid	Invalid receiver	FAILED	sender
R03	balance too low	Insufficient balance	FAILED	sender
R04	debit ERROR	Sender bank declined	FAILED	sender
R05	debit TIMEOUT	Sender bank silent	TIMEOUT	sender, ops
R06	debit OK, credit OK	Normal success	COMPLETED	none
R07	debit OK, credit ERROR	Receiver rejected credit	RECONCILIATION_REQUIRED	all three
R08	debit OK, credit TIMEOUT, status check: credited	Credit applied, confirmation lost	COMPLETED	sender, receiver
R09	debit OK, credit TIMEOUT, status check: not credited	Receiver did not credit	RECONCILIATION_REQUIRED	all three

Rule	Signals	Reason	Next state	Alerts
R10	credit TIMEOUT, no check possible, bank down	Receiver bank unavailable	TIMEOUT	all three
R11	credit TIMEOUT, no check possible, network down	Network failure	TIMEOUT	sender, ops
R12	credit TIMEOUT, no check yet	Receiver slow	TIMEOUT	sender
R99	anything else	Unknown combination	RECONCILIATION_REQUIRED	ops

Every row also returns an **action**, a plain-language **user message** and a **severity**. Rows with money taken but not delivered tell the sender not to retry. The engine itself is small:

```
# Rule-based failure diagnosis. A table, not a pile of if/else, so it can be
# printed as the decision table in the report. First matching row wins.
# Each row: (rule id, signals to match, outcome)

DEFAULTS = {"duplicate": False, "receiver_valid": True, "balance_ok": True,
            "debit": None, "credit": None, "status_check": None,
            "network_ok": True, "bank_up": True}

# (the RULES list holds one tuple per row: id, signals to match, outcome)

def diagnose(signals):
    s = {**DEFAULTS, **signals}
    for rule_id, pattern, outcome in RULES:
        if all(s.get(k) == v for k, v in pattern.items()):
            return {"rule": rule_id, **outcome}
    return {"rule": "R99", **FALLBACK}
```

The reference table passed all 13 test cases: the 10 payment scenarios, the debit-timeout case, a split of the credit-timeout case into with and without a status check, and an unknown combination. Limitation to state in the viva: the system cannot know a bank's internal reason unless the bank exposes it, so reasons are inferred from observable signals only.

Step 5. Recovery-time estimator

Group past stuck payments by diagnosed reason and report the typical resolution time for that reason. Start with medians and a typical range. If there are too few similar cases, say so instead of guessing.

```
from collections import defaultdict
from statistics import median, quantiles

MIN_SAMPLES = 5


def build_table(history):
    """history: list of {"reason": str, "minutes": float} from past stuck cases."""
    groups = defaultdict(list)
    for h in history:
        groups[h["reason"]].append(h["minutes"])
    table = {}
    for reason, mins in groups.items():
        if len(mins) >= 2:
            q = quantiles(mins, n=4)              # [p25, p50, p75]
            low, high = q[0], q[2]
        else:
            low = high = mins[0]
        table[reason] = {"median": median(mins), "low": low, "high": high, "n": len(mins)}
    return table


def estimate(reason, table):
    row = table.get(reason)
    if row is None or row["n"] < MIN_SAMPLES:
```

```
        return {"minutes": None, "confidence": "LOW",
                "message": "Not enough similar cases yet. We will update you when we know."}
    mid, low, high = round(row["median"]), round(row["low"]), round(row["high"])
    return {"minutes": mid, "range": (low, high), "confidence": "NORMAL", "based_on": row["n"],
            "message": f"Cases like this are usually resolved in about {mid} minutes "
                       f"(typically {low} to {high}). This is an estimate, not a guarantee."}
```

The estimator is only as good as its history. While developing, use a generated sample and label it as such. For the report, use resolution times that come from Member 1's simulator runs, and never present the estimate as a promise.

Step 6. Dataset and models

No real UPI data is public, so generate a synthetic labelled dataset. Decide fraud or normal first (about 0.8 percent fraud), then draw each feature from a different but overlapping distribution for the two groups.

Feature	Type	How fraud differs
amount_ratio	number	amount divided by the customer's usual; higher
hour	0-23	more often at night
is_new_device, is_new_receiver	0 or 1	much more often 1
txn_count_1h, failed_attempts_24h	count	higher
distance_km	number	farther from usual location
merchant_category, device_type, channel	category	different mix (gift cards, web more common)
is_fraud	0 or 1	label to predict

```
import numpy as np
import pandas as pd

CATS = ["grocery", "food", "bills", "travel", "electronics", "gift_cards"]

def make_dataset(n=100000, fraud_rate=0.008, seed=42):
    """Synthetic data: decide fraud or normal first, then draw each feature from a
    different (overlapping) distribution for the two groups."""
    rng = np.random.default_rng(seed)
    fraud = rng.binomial(1, fraud_rate, n).astype(bool)
    f, g = fraud, ~fraud
    nf, ng = f.sum(), g.sum()

    def mix(fraud_draw, normal_draw):
        out = np.empty(n)
        out[f] = fraud_draw(nf)
        out[g] = normal_draw(ng)
        return out

    hour = np.empty(n, dtype=int)
    night = rng.random(nf) < 0.55                  # fraud skews to the night
    hour[f] = np.where(night, rng.integers(0, 5, nf), rng.integers(5, 24, nf))
    hour[g] = rng.choice(np.arange(24), ng, p=_day_profile())

    df = pd.DataFrame({
```

Train a baseline (Logistic Regression) and the final model (CatBoost). CatBoost takes categorical columns directly, while Logistic Regression needs them one-hot encoded. Both use class weights because fraud is rare.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (roc_auc_score, average_precision_score, recall_score,
```

```
                              precision_score, f1_score, confusion_matrix)
from catboost import CatBoostClassifier
from ml.make_data import make_dataset

CAT = ["merchant_category", "device_type", "channel"]
df = make_dataset()
X, y = df.drop(columns="is_fraud"), df["is_fraud"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y,
                                      random_state=0)

def report(name, proba):
    pred = (proba >= 0.5).astype(int)
    tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
    print(f"{name:19s} ROC-AUC {roc_auc_score(yte, proba):.3f}  "
          f"PR-AUC {average_precision_score(yte, proba):.3f}  "
          f"recall {recall_score(yte, pred):.2f}  precision {precision_score(yte, pred):.2f}  "
          f"F1 {f1_score(yte, pred):.2f}  TP {tp} FP {fp} FN {fn} TN {tn}")

# Baseline: logistic regression (categoricals one-hot encoded)
Xtr_oh = pd.get_dummies(Xtr, columns=CAT)
Xte_oh = pd.get_dummies(Xte, columns=CAT)
Xte_oh = Xte_oh.reindex(columns=Xtr_oh.columns, fill_value=0)
lr = make_pipeline(StandardScaler(), LogisticRegression(class_weight="balanced", max_iter=1000))
lr.fit(Xtr_oh, ytr)

# Final: CatBoost (native categorical handling)
cb = CatBoostClassifier(iterations=200, depth=4, learning_rate=0.1,
                        auto_class_weights="Balanced", cat_features=CAT,
                        verbose=0, random_seed=0)
cb.fit(Xtr, ytr)

print("train rows", len(Xtr), "test rows", len(Xte), "test fraud", int(yte.sum()))
report("LogisticRegression", lr.predict_proba(Xte_oh)[:, 1])
report("CatBoost", cb.predict_proba(Xte)[:, 1])
imp = sorted(zip(cb.feature_names_, cb.get_feature_importance()), key=lambda t: -t[1])[:5]
print("top features:", [(n, round(v, 1)) for n, v in imp])
cb.save_model("fraud_model.cbm")
```

Reference run: 100,000 rows, 0.8% fraud, 20,000 test rows (160 fraud)	ROC-AUC	PR-AUC	Recall	Precision	F1
Logistic Regression (baseline)	0.999	0.937	0.98	0.29	0.44
CatBoost (final)	0.998	0.938	0.97	0.43	0.59

Read these numbers carefully. Both models score almost perfectly because the synthetic data is easy: fraud and normal rows are generated to look quite different. That proves the pipeline works, not that real fraud is this easy. CatBoost's real advantage here is fewer false alarms at the same recall (precision 0.43 against 0.29). If your AUC is above 0.99, make the groups overlap more before you report, and always state that results come from synthetic data.

Report recall, precision, F1, the confusion matrix, ROC-AUC and PR-AUC. Do not report plain accuracy, because a model that always says "not fraud" would already be 99 percent accurate.

Step 7. Blend the model into scoring and integrate

The model needs features built from the transaction and customer context. If no trained model file exists the system falls back to rules only, which keeps the project working early. The final score blends both:

```
final score = 0.6 x weighted rule score + 0.4 x (model probability x 100)

import os
import pandas as pd
from catboost import CatBoostClassifier

ORDER = ["amount_ratio", "hour", "is_new_device", "is_new_receiver", "txn_count_1h",
         "failed_attempts_24h", "distance_km", "merchant_category", "device_type", "channel"]
_model = None
```

```
HERE = os.path.dirname(__file__)


def load_model(path=os.path.join(HERE, "..", "fraud_model.cbm")):
    global _model
    if _model is None and os.path.exists(path):
        _model = CatBoostClassifier(); _model.load_model(path)
    return _model


def build_features(txn, ctx):
    return {"amount_ratio": txn["amount"] / max(ctx["usual_amount"], 1),
            "hour": txn["hour"],
            "is_new_device": int(txn.get("device_id") not in ctx["known_devices"]),
            "is_new_receiver": int(txn.get("receiver_id")
                                   not in ctx.get("known_receivers", set())),
            "txn_count_1h": ctx.get("count_1h", ctx["recent_count"]),
            "failed_attempts_24h": ctx.get("failed_attempts", 0),
            "distance_km": ctx.get("distance_km", 0.0),
            "merchant_category": ctx.get("merchant_category", "grocery"),
            "device_type": ctx.get("device_type", "android"),
            "channel": ctx.get("channel", "app")}


def ml_probability(txn, ctx):
    """Fraud probability from the model, or None if no trained model file exists."""
    model = load_model()
    if model is None:
        return None
    row = pd.DataFrame([build_features(txn, ctx)])[ORDER]
    return float(model.predict_proba(row)[0, 1])
```

Reference results with both layers: an ordinary 1200 payment from a known device and receiver scored 0.6 (APPROVE). A 9500 payment at 3 am from a new city, new device, new receiver, gift-card merchant, six recent transactions and three failed attempts scored 83.2 (rule score 72, model 100) and was BLOCK. The 0.6 / 0.4 split and the thresholds are a starting point; confirm them with the guide and tune on your data.

Note: the model probabilities come from a model trained with class weights, so they are not calibrated probabilities. Treat the model score as a ranking signal, not as a literal percentage.

Step 8. Hardening and the results section

- [ ] Write pytest tests for every case in section 8.
- [ ] Re-run the benchmark and the model evaluation on the final code and save the numbers.
- [ ] Check the output fields match what Member 3's dashboard expects: decision, score, per-check reason, diagnosis reason, severity, action, recovery message.
- [ ] Write the results section of the report: benchmark table, model comparison table, decision table, limitations.

6. What You Give and Receive

With	You give	You receive
Member 1	score_transaction(txn); diagnose(signals); estimate(reason); a trained model file	Transaction data, customer context, diagnosis signals from the simulators, recovery history logs
Member 3	Per-check scores and reasons; diagnosis reason, severity, action and user message; recovery message	Questions about field names and wording
Guide	Pull requests; benchmark and model results	Sign-off at each checkpoint

Unblock Member 1 first: deliver a working sequential score_transaction and diagnose() early, even before the thread pool and model exist.

7. How the Parts Connect

Moment in a payment	Member 1 calls	You return	Used for
Before the debit	score_transaction(txn)	APPROVE, REVIEW or BLOCK with a score and per-check reasons	Stop the payment or open a case
A step fails or times out	diagnose(signals)	Reason, next state, action, message, severity, who to alert	Choose the next state and write alerts
After diagnosing a stuck payment	estimate(reason)	Estimated minutes with a typical range, or not enough data	Message to the sender and the dashboard

8. What You Must Pass

#	Test	Expected result
1	Ordinary payment from known device, usual amount	APPROVE
2	Large payment, new city, 3 am, unknown device	REVIEW or BLOCK, with a reason on each check
3	Six checks started together	Start times within a few milliseconds; six different thread names
4	Benchmark on the same batch	Concurrent clearly faster; numbers recorded
5	Diagnose: normal success	COMPLETED
6	Diagnose: credit error	RECONCILIATION_REQUIRED, alerts to sender, receiver, ops
7	Diagnose: credit timeout, status check shows credit	COMPLETED (confirmation lost)
8	Diagnose: credit timeout, status check shows no credit	RECONCILIATION_REQUIRED
9	Diagnose: debit error, invalid receiver, low balance	FAILED, nothing deducted
10	Diagnose: duplicate, network failure, bank down	UNCHANGED, TIMEOUT, TIMEOUT
11	Diagnose: unknown signal combination	RECONCILIATION_REQUIRED with ops alert
12	Recovery estimate with too few cases	Says not enough data; no invented number
13	Model file missing	Scoring still works from rules only
14	Model evaluation	Recall, precision, F1, ROC-AUC reported; limits of synthetic data stated

9. Rules to Follow

Do

- Keep diagnosis free of machine learning. Its value is being instant and explainable with no training data.
- Keep checks read-only and free of shared state so threads cannot interfere.
- Keep the weights and thresholds in one place so they are easy to tune and explain.

- Always show estimates as estimates.
- Open a pull request for every change and ask Member 1 or 3 to review it.

Do not

- Change a payment's state yourself. Return a recommended next state and let Member 1 apply it.
- Share one database session across threads.
- Report accuracy alone, or claim the model works on real UPI data.
- Say the six threads detect the fault. They score fraud risk.
- Edit another member's folder. Open an issue instead.

10. Definition of Done

Phase 0

[ ] Signatures, context fields, signal names and BLOCK/REVIEW handling agreed in writing with Member 1.

Phase 1

[ ] Six checks return a score and reason.

[ ] score_transaction works sequentially.

[ ] diagnose() passes all its tests.

Phase 2

[ ] Member 1's flow calls score_transaction and acts on the decision.

[ ] Member 3 can display your fields.

Phase 3

[ ] Thread pool works; thread names and times are logged.

[ ] Benchmark numbers recorded.

[ ] Recovery estimator works on simulator history.

[ ] Dataset generated; both models trained and evaluated.

Phase 4

[ ] Model blended into scoring, with rules-only fallback.

[ ] All tests in section 8 pass.

[ ] Benchmark and evaluation re-run on final code.

[ ] Results section drafted for the report.

11. Questions You Should Be Able to Answer

Question	Answer
What do the six threads do?	They score fraud risk at the same time. They do not find the payment fault; the rule table does.
Why threads and not processes?	The checks mostly wait on lookups (I/O-bound). Threads overlap waiting cheaply; processes help CPU-heavy work.
How do you prove concurrency helps?	Same batch run sequentially and concurrently: about 700 ms against about 200 ms per payment in the reference run.
Do threads need locks here?	No. Each check only reads data and returns its own result. Duplicate protection is Member 1's database constraint.
How do you find the failure reason?	A decision table over real-time signals. First matching row gives reason, state and action. No history needed.
Why not use ML for diagnosis?	It must be instant and explainable, and needs no training data. A rule table does that. Papers on payment failure detection take the same approach.
Where is ML used?	Fraud scoring and, if wanted, the recovery estimate. Both depend on historical patterns.
Why CatBoost, and what is your baseline?	CatBoost handles categorical columns directly and copes with rare fraud. Baseline is Logistic Regression.
Why not accuracy?	Fraud is under 1 percent, so always answering no fraud is 99 percent accurate. Use recall, precision, F1 and AUC.
Which dataset?	Synthetic, because no real UPI data is public. It demonstrates the method, not real-world accuracy.
Is the recovery time a promise?	No. It is an estimate from similar past cases and is shown with a typical range.

12. Common Mistakes to Avoid

- Sharing one database session between threads.
- Putting a model inside the diagnosis engine.
- Reporting great results from a dataset that is too easy without saying so.
- Tuning thresholds on the same rows you report results on.
- Hardcoding the weights in several places.
- Waiting for others. Deliver a simple working version first and improve it.